

Современная реализация архитектуры трансформера заметно отличается от оригинальной версии. Более того, каждый год исследователи по всему миру стараются ее улучшить.

Главные причины этих изменений — сугубо практические. В основном все они направлены на улучшение масштабирования модели и на большую стабильность обучения. Мы хотим лучше использовать GPU во время обучения, мы хотим делать модели больше, глубже, обрабатывать больший контекст и при этом избегать чрезмерного роста вычислений и памяти.

1 Улучшение архитектуры трансформеров

1.1 Pre-norm, Post-norm и RMSNorm

В оригинальной архитектуре трансформера после слоя внимания или FFN-блока выполняется сложение с остаточной связью, а затем нормализация.

Такой вариант называется **пост-нормализацией** (или **Post-norm**):

$$y = \text{LayerNorm}(x + f(x))$$

Пост-нормализация позволяет достичь большей численной стабильности в остаточных связях.

Остаточные связи в свою очередь нужны для стабильного распространения градиентов. Если проводить аналогию, что ребра графа вычислений — это дороги, то residual connections — это магистраль для градиентов.

Однако, ранее мы уже дифференцировали слой трансформера с пост-нормализацией:

$$\frac{\partial y}{\partial x} = \underbrace{\frac{\partial \text{LN}(\tilde{z})}{\partial \tilde{z}} \frac{\partial \text{LN}(\tilde{x})}{\partial \tilde{x}}}_{\text{Градиенты почти не затухают}} \cdot \left(1 + \underbrace{\frac{\partial \text{FFN}(z)}{\partial z} + \frac{\partial \text{MHA}(x)}{\partial x}}_{\text{Градиенты затухают слабее}} + \underbrace{\frac{\partial \text{FFN}(z)}{\partial z} \cdot \frac{\partial \text{MHA}(x)}{\partial x}}_{\text{Градиенты сильно затухают}} \right),$$

здесь y — выход слоя трансформера, а x — вход.

Да, градиенты при такой архитектуре почти не затухают на одном слое. Но что, если мы захотим построить архитектуру с, например, 64 слоями? Тогда на нашей ‘магистрали’ градиентов после каждого слоя появятся КПП, которых хотелось бы избежать.

Чтобы избежать этой проблемы, можно изменить порядок операций:

$$y = x + f(\text{LayerNorm}(x))$$

В этом случае, нормализация будет находиться внутри слоя, а при дифференцировании не будут возникать дополнительные множители, которые приведут к затуханию градиентов. Такая нормализация называется **пре-нормализацией (Pre-norm)**.

Ранее мы обсуждали, что хотели бы хранить активации в пониженной точности (в **bf16** если быть точнее). Однако, убрав нормализации над активациями в остаточных связях, мы рискуем прийти к численной нестабильности обучения.

Так например, активации могут вырасти слишком сильно, что приведет к большим ошибкам округления. В таком случае, мы можем поставить нормализацию в начале (нам нужна нормализация перед вычислением блока) и в конце преобразования:

$$y = x + \text{LayerNorm}\left[f(\text{LayerNorm}(x))\right]$$

Такой подход называется **сэндвич-нормализацией (Sandwich-norm)**. Однако и он несет издержки. Нормализация, как мы помним, имеет низкую арифметическую интенсивность и при ее вычислении необходимо выполнить две операции редукции для вычисления статистик.

К тому же, при использовании тензорного параллелизма и параллелизма последовательностей могут возникнуть трудности с включением нормализации в эпилог более крупного ядра.

При частом применении в модели это грозит стать бутылочным горлышком и снизить утилизацию GPU. Эти проблемы в некоторой степени решает **RMSNorm**:

$$\text{RMSNorm}(x) = \gamma \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \varepsilon}}$$

В отличие от LayerNorm, RMSNorm не вычисляет среднее и подгружает немного меньше параметров.

1.2 Gated FFN

В оригинальной архитектуре трансформера FFN-блок устроен как два линейных слоя с нелинейной функцией активации между ними

$$\text{FFN}(x) = W_2 f(x W_1),$$

где f — это, например, ReLU или GeLU.

Идея ReLU, LeakyReLU, GeLU и им подобным функциям — не просто добавить нелинейность, а создать шлюз, который регулирует прохождение информации.

Если активация большая — ее информация проходит дальше. Если малая — то зануляется (или сильно уменьшается). Возможно, мы бы смогли значительно повысить выразительную силу модели, если бы сделали механизм шлюза — обучаемым.

Такие функции активации называются **GLU (Gated Linear Unit)**. В общем случае они выглядят так:

$$\text{GLU}(x) = f(x W_1) \otimes (x W_2),$$

где $\otimes$ — поэлементное произведение, а f — функция активации.

Одна ветка вычисляет признаки, а другая ветка управляет тем, какие из этих признаков пройдут дальше. Поэтому GLU лучше воспринимать не как отдельную функцию активации, а как структуру слоя.

В современных архитектурах часто используется вариант с функцией SiLU:

$$\text{SiLU}_{\beta=1}(x) = x \sigma(x)$$

Такая функция активации называется SwiGLU:

$$\text{SwiGLU}(x) = \text{SiLU}(xW_1) \otimes (xW_2)$$

После результат надо пропустить через выходную линейную проекцию:

$$\text{FFN}_{\text{SwiGLU}}(x) = (\text{SiLU}(xW_1) \otimes xW_2)W_3$$

Из-за дополнительной линейной проекции число параметров в FFN-блоке увеличивается. Чтобы сохранить примерно тот же параметрический бюджет, внутреннюю размерность обычно уменьшают. Поэтому вместо классического расширения $d \rightarrow 4d \rightarrow d$ используют меньшую внутреннюю размерность для $\text{FFN}_{\text{SwiGLU}}$ $d_{\text{FFN}} = \frac{8}{3}d$.

Мы ранее обсуждали проблему увеличения объема хранимых активаций. На первый взгляд, SwiGLU заставит нас хранить вдвое больше активаций на входной проекции FFN.

Однако, мы можем вычислять всё SwiGLU в одном ядре, что позволит сократить количество хранимых промежуточных значений.

Более того, при малой размерности d мы можем вычислять даже весь $\text{FFN}_{\text{SwiGLU}}$. Это позволит выиграть по памяти, однако позволит ли выиграть по вычислениям — вопрос более тонкий.

1.3 Улучшение позиционного кодирования

Механизм внимания сам по себе, как мы уже обсуждали при разборе оригинальной архитектуры трансформера, не знает порядок токенов. Он смотрит параллельно на все токены как на неупорядоченное множество.

Мы уже рассматривали абсолютное синусоидальное позиционное кодирование, которое используется в оригинальном трансформере. А также — абсолютное обучаемое позиционное кодирование из Vision Transformer’a.

У абсолютного позиционного кодирования есть очевидное ограничение. Модель получает информацию о конкретном номере позиции, но для многих задач важнее не абсолютный номер, а относительное расположение токенов.

Обычно модели важнее знать, что токен t_i стоит перед токеном t_{i+1} , а не то, что один токен имеет абсолютную позицию, например, 239, а другой — 240.

Рассмотрим методы **относительного позиционного кодирования**. В этом случае позиционная информация зависит от пары токенов и их относительного расстояния.

В классическом внимании элемент матрицы оценок внимания имеет вид:

$$\text{score}(i, j) = \frac{Q_i K_j^T}{\sqrt{d}} = \frac{x_i W_Q (x_j W_K)^T}{\sqrt{d}}$$

В относительном позиционном кодировании к ключу (или к оценке внимания) добавляется информация о расстоянии между позициями i и j :

$$\text{score}(i, j) = \frac{x_i W_Q (x_j W_K + a_{ij})^T}{\sqrt{d}},$$

где a_{ij} — вектор, некоторым образом кодирующий относительное положение токена j относительно токена i .

Такой подход лучше отражает структуру текста, потому что модель начинает учитывать не только содержимое токенов, но и расстояние между ними.

Один из наиболее важных современных вариантов позиционного кодирования — **RoPE** (Rotary Position Embedding).

RoPE можно воспринимать как гибридную идею. С одной стороны, позиция задается через абсолютный номер p .

С другой стороны, при вычислении скалярного произведения запросов и ключей возникает зависимость от относительного расстояния между позициями.

В RoPE вектор разбивается на пары координат, и каждая пара поворачивается на угол, зависящий от позиции. Для вектора запроса q_p на позиции p можно записать:

$$R(q, p) = \begin{pmatrix} q_1 \\ q_2 \\ \dots \\ q_d \end{pmatrix}^\top \begin{bmatrix} M_1 & 0 & \dots & 0 \\ 0 & M_2 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & \dots & 0 & M_{d/2} \end{bmatrix},$$

где каждый блок M_k — матрица поворота для пары координат:

$$M_k = \begin{pmatrix} \cos(p\omega_k) & \sin(p\omega_k) \\ -\sin(p\omega_k) & \cos(p\omega_k) \end{pmatrix}$$

Частоты ω_k выбираются аналогично синусоидальному позиционному кодированию:

$$\omega_k = \frac{1}{10000^{2(k-1)/d}}$$

Тогда оценка внимания с RoPE принимает вид:

$$\text{score}(i, j) = \frac{R(x_i W_Q, i) R(x_j W_K, j)^T}{\sqrt{d}}$$

Вычислять $R(q, p)$ через умножение на большую блочно-диагональную матрицу невыгодно, потому что эта матрица почти вся состоит из нулей. На практике применяют поэлементную формулу для пар координат:

$$\hat{q}_{2k-1} = q_{2k-1} \cos(p\omega_k) - q_{2k} \sin(p\omega_k),$$

$$\hat{q}_{2k} = q_{2k-1} \sin(p\omega_k) + q_{2k} \cos(p\omega_k),$$

Теперь посмотрим, почему RoPE содержит относительную информацию. Для простоты рассмотрим пару координат как комплексное число. Тогда поворот можно записать как умножение на комплексную экспоненту:

$$q_p \xrightarrow{\text{RoPE}} q \cdot e^{ip\omega}.$$

Если один и тот же вектор находится на позициях p и l , то получаем:

$$\hat{q}_p = q \cdot e^{ip\omega}, \quad \hat{q}_l = q \cdot e^{il\omega}.$$

Переход от одной позиции к другой задается множителем

$$e^{i(p-l)\omega}.$$

Он зависит не от абсолютных позиций по отдельности, а от их разности $p - l$. Вот здесь и находится информация об относительной позиции.

Вычислять каждый раз косинусы и синусы — слишком дорого. Имеет смысл заранее подготовить кэш RoPE, а именно косинусы и синусы для разных позиций.

1.4 Улучшение остаточных связей

Способность нейронных сетей распознавать сложные паттерны в данных обусловлена наличием нелинейных функций активации. Более глубокие модели способны распознавать более сложные представления, чем менее глубокие.

Чтобы избежать проблемы затухающих градиентов, мы используем остаточные связи (residual connections)

$$y = x + f(x)$$

Но такой подход достаточно ограничен, поскольку предполагает только одну связь. А что, если мы будем использовать несколько таких связей?

1.4.1 Гиперсвязи

Сказано — сделано. Увеличим количество связей до R . Но обрабатывать каждую такую связь блоком трансформера — дорого.

Просуммируем их, используя взвешенную сумму:

$$\hat{x} = \sum_{i=1}^R h_i^{\text{pre}} x_i,$$

где h_i^{pre} — обучаемый параметр.

Этот вход обработаем блоком слоёв:

$$\hat{z} = f(\hat{x}),$$

где f , может быть, например, блок FFN или МНА.

Выход такого блока — всего один, а остаточных связей — R . В таком случае, введем обучаемые параметры h_i^{post} для масштабирования блока под остаточные связи:

$$z_i = h^{\text{post}} \hat{z}$$

И прибавим этот результат к остаточной связи:

$$y_i = x_i + z_i$$

Чтобы увеличить обмен информацией между остаточными связями, добавим взвешенное среднее по остаточным связям:

$$y_i = \sum_{j=1}^R h_j^{\text{res}} x_j + z_i,$$

где h_j^{res} — также обучаемый параметр.

Перепишем в матричном виде (в col-major записи, это упростит нам запись в будущем):

$$\hat{x} = \sum_{i=1}^R h_i^{\text{pre}} x_i = H^{\text{pre}} \hat{z},$$

где

$$H^{\text{pre}} = [h_1^{\text{pre}} \quad h_2^{\text{pre}} \quad \dots \quad h_R^{\text{pre}}]$$

— матрица агрегации остаточных связей.

Идем дальше:

$$z = \begin{bmatrix} h_1^{\text{post}} \hat{z} \\ h_2^{\text{post}} \hat{z} \\ \dots \\ h_R^{\text{post}} \hat{z} \end{bmatrix} = H^{\text{post}} \hat{z}$$

Здесь H^{post} — матрица расширения выходов блока.

И аналогично:

$$y = H^{\text{res}} x + z,$$

где H^{res} — матрица смешивания остаточных связей.

Соберем всё вместе:

$$y = H^{\text{res}} x + H^{\text{post}} f[H^{\text{pre}} x]$$

Такой подход называется **гиперсвязи (hyper-connections)**.

1.4.2 Динамические гиперсвязи

H^{res} , H^{post} и H^{pre} — обучаемые матрицы на каждом слое. Но они не адаптивны к входным данным. Пока что — это просто линейное отображение.

Но что, если мы заменим эти матрицы на более сложные преобразования, добавив нелинейность:

$$H^{\text{pre}}(x) = \alpha^{\text{pre}} \tanh(\theta^{\text{pre}} x^\top) + b^{\text{pre}},$$

$$H^{\text{post}}(x) = \alpha^{\text{post}} \tanh(\theta^{\text{post}} x^\top) + b^{\text{post}},$$

$$H^{\text{res}}(x) = \alpha^{\text{res}} \tanh(\theta^{\text{res}} x^\top) + b^{\text{res}}$$

Обучаемые параметры в таком случае можно разделить на 2 группы:

- параметры статических преобразований, которые не зависят от входных данных (например, α^{pre} и b^{pre}),
- параметры динамических преобразований, зависящие от входных данных (например, θ^{pre})

Такой подход называется **динамические гиперсвязи (Dynamic Hyper-Connections)**. Однако, у него есть большая проблема — нестабильность обучения.

Напишем выражение для динамических связей блока f_l слоя l :

$$x_{l+1} = H_l^{\text{res}}(x_l) + H_l^{\text{post}}\left(f_l\left[H_l^{\text{res}}(x_l)\right]\right)$$

Сейчас запись слишком сложная, упростим ее. Представим H^{res} , H^{post} и H^{pre} как операторы преобразования. Тогда мы можем переписать это выражение следующим образом:

$$x_{l+1} = H_l^{\text{res}} x_l + H_l^{\text{post}} f_l \left[H_l^{\text{res}} x_l \right]$$

Выполним преобразования дальше, до финального слоя L :

$$x_L = \left(\prod_{i=1}^{L-l} H_{L-i}^{\text{res}} \right) x_l + \sum_{i=l}^{L-1} \left(\prod_{j=1}^{L-1-i} H_{L-j}^{\text{res}} \right) H_i^{\text{post}} f \left(H_i^{\text{pre}} x_i \right),$$

где $\left(\prod_{i=1}^{L-l} H_{L-i}^{\text{res}} \right)$ и $\left(\prod_{j=1}^{L-1-i} H_{L-j}^{\text{res}} \right)$ — комплексные преобразования, полученные в результате последовательного применения операторов смешивания остаточных связей.

Сравним с обычными остаточными связями:

$$x_L = x_l + \sum_{i=l}^L f_i(x_i)$$

Первое слагаемое в обычных остаточных связях позволяет градиентам беспрепятственно распространяться к каждому слою.

Но в случае с гиперсвязями у нас появляется преобразование $\left(\prod_{i=1}^{L-l} H_{L-i}^{\text{res}} \right)$. Поскольку значения никак не ограничены, это может привести к взрыву активаций (не говоря уже об очевидных проблемах с градиентами).

Особенно плохо дела будут обстоять, если эти преобразования будут менять знак активаций. В наших же интересах, особенно при обучении в малой точности, чтобы активации оставались стабильными.

Возвращаясь к аналогии с магистралью, мы снова добавили блокпосты, которые будут мешать активациям и градиентам.

1.4.3 Manifold-Constrained Hyper-Connections (mHC)

Гиперсвязи очень полезны, а динамические гиперсвязи расширяют выразительную силу модели и добавляют дополнительные нелинейности. Не хотелось бы отказываться от их использования.

Поэтому необходимо обеспечить корректную работу операторов H^{res} , H^{post} и H^{pre} , чтобы избежать нестабильности. Для того, чтобы сделать это, для начала упростим себе задачу и на время откажемся от использования динамических гиперсвязей.

В первую очередь необходимо некоторыми образом ограничить параметры H^{res} , чтобы стабилизировать активации остаточных связей во время обучения.

Для этого сделаем эту матрицу дважды стохастической: необходимо, чтобы все элементы были положительными, а сумма по каждой строке и каждому столбцу должна быть равна 1.

Чтобы обеспечить положительность, применим экспоненту к каждому элементу.

Чтобы получить дважды стохастическую матрицу, будем итеративно масштабировать столбцы и строки. Через ограниченное число итераций мы получим матрицу, близкую к дважды стохастической. Этот алгоритм называется **алгоритмом Синхрона-Кноппа (Sinkhorn-Knopp)**.

Теперь вернемся к динамическим связям. H^{res} преобразование мы можем записать следующим образом:

$$H_l^{\text{res}} = \text{Sinkhorn-Knopp} \left[\exp \left(\alpha_l^{\text{res}} \text{mat}(x_l \varphi_l^{\text{res}}) + b_l^{\text{res}} \right) \right],$$

где mat — обратная операция к операции "выпрямления" матрицы в вектор. Нелинейность преобразования, роль которой ранее выполняла функция $\tanh$, теперь выполняет экспонента.

Рассмотрим H^{post} и H^{pre} :

$$H^{\text{pre}} = \sigma \left(\alpha_l^{\text{pre}} (x_l \varphi_l^{\text{pre}}) + b_l^{\text{pre}} \right)$$

$$H^{\text{post}} = 2\sigma \left(\alpha_l^{\text{post}} (x_l \varphi_l^{\text{post}}) + b_l^{\text{post}} \right)$$

Во-первых, мы заменили нелинейность на сигмоиду. Во-вторых, мы вынесли нелинейность из внутренней части выражения во внешнюю. Таким образом, мы гарантируем положительность H^{post} и H^{pre} .

В H^{post} параметры α_l^{post} и b_l^{post} в начале обучения близки к нулю. Это означает, что аргумент сигмиды будет близок к нулю, а это в свою очередь означает, что значение сигмиды будет равно 0.5. Мы домножаем на 2, чтобы приблизить его к 1.

Тогда в начале обучения гиперсвязи будут в некотором смысле повторять поведение обычных остаточных связей, что благоприятно скажется на стабильности обучения на первых шагах.

Соберем всё вместе:

$$\begin{cases} H^{\text{pre}} = \sigma(\alpha_l^{\text{pre}}(x_l \varphi_l^{\text{pre}})) + b_l^{\text{pre}}, \\ H^{\text{post}} = 2\sigma(\alpha_l^{\text{post}}(x_l \varphi_l^{\text{post}})) + b_l^{\text{post}}, \\ H_l^{\text{res}} = \text{Sinkhorn-Knopp} \left[\exp(\alpha_l^{\text{res}} \text{mat}(x_l \varphi_l^{\text{res}}) + b_l^{\text{res}}) \right] \end{cases}$$

Такие гиперсвязи называются **ограниченные в многообразии гиперсвязи** (Manifold-Constrained Hyper-Connections, mHC)

2 Эффективное внимание

FlashAttention избавляет нас от большой проблемы — квадратичного роста занимаемой памяти от длины последовательности. Однако, квадратичный рост количества вычислений всё ещё остается достаточно крупной проблемой.

2.1 KV Cache

Начнем с инференса. Модель генерирует новую последовательность авто-регрессионно. Вычислять квадратичное внимание в таком случае было бы очень дорого.

На каждом шаге модель получает уже сгенерированные токены и должна предсказать следующий токен.

Для входной матрицы X_t вычисляются матрицы запросов, ключей и значений:

$$Q_t = X_t W_Q, \quad K_t = X_t W_K, \quad V_t = X_t W_V$$

Далее вычисляется матрица оценок внимания:

$$S_t = Q_t K_t^T,$$

после чего применяется маска и softmax:

$$A_t = \text{Softmax} \left(\text{Mask} \left[\frac{S_t}{\sqrt{d}} \right] \right)$$

Выход внимания получается умножением весов внимания на матрицу значений:

$$O_t = A_t V_t$$

Теперь рассмотрим следующий шаг генерации. Вход X_{t+1} отличается от X_t только тем, что к последовательности добавился один новый токен:

$$X_{t+1} = \begin{bmatrix} X_t & x_{t+1} \end{bmatrix}$$

Тогда матрицы запросов, ключей и значений можно переписать следующим образом:

$$Q_{t+1} = X_{t+1} W_Q = \begin{bmatrix} X_t \\ x_{t+1} \end{bmatrix} W_Q = \begin{bmatrix} Q_t \\ x_{t+1} W_Q \end{bmatrix},$$

$$K_{t+1} = \begin{bmatrix} K_t \\ x_{t+1} W_K \end{bmatrix}, \quad V_{t+1} = \begin{bmatrix} V_t \\ x_{t+1} W_V \end{bmatrix}$$

Видно, что большая часть этих матриц уже была вычислена на предыдущем шаге. Новыми являются только векторы для последнего токена.

Кроме того, при авторегрессионной генерации используется треугольная маска. Новый токен смотрит на все предыдущие токены, но предыдущие токены его не видят. Поэтому при добавлении нового токена фактически нужно вычислять только новую строку матрицы внимания.

Идея **KV-кэш** проста: не пересчитывать ключи и значения для уже обработанных токенов.

На новом шаге вычисляем только новые векторы:

$$q_{t+1} = x_{t+1} W_Q, \quad k_{t+1} = x_{t+1} W_K, \quad v_{t+1} = x_{t+1} W_V$$

Старые ключи и значения берем из кэша:

$$K_{t+1} = \begin{bmatrix} K_{\text{cached}} \\ k_{t+1} \end{bmatrix}, \quad V_{t+1} = \begin{bmatrix} V_{\text{cached}} \\ v_{t+1} \end{bmatrix}$$

После этого кэш обновляется:

$$\tilde{K}_{\text{cached}} = \begin{bmatrix} K_{\text{cached}} \\ k_{t+1} \end{bmatrix}, \quad \tilde{V}_{\text{cached}} = \begin{bmatrix} V_{\text{cached}} \\ v_{t+1} \end{bmatrix}$$

Теперь вместо пересчета всех K и V на каждом шаге мы только дописываем один новый вектор. Это резко уменьшает количество вычислений при генерации.

Однако KV Cache переносит проблему из вычислений в память. Для каждого слоя и для каждой головы нужно хранить ключи и значения всех уже обработанных токенов. Объем KV-кэша можно оценить так:

$$\text{size(KV-cache)} = \text{batch_size} \times \text{seq_length} \times N_l \times N_h \times 2 \times d_{\text{head}} \times \text{dtype_size}$$

Множитель 2 возникает из-за того, что хранятся и ключи, и значения. При длинном контексте KV Cache потребляет очень много памяти и это проблема.

Иногда имеет смысл часть KV-кэша выносить из памяти GPU в оперативную память хоста или даже на диск. Но это инженерное решение.

2.2 MQA, GQA

Рассмотрим архитектурные решения проблемы большого KV-кэша.

Основная причина в том, что в обычном multi-head attention для каждой головы хранятся свои матрицы K и V .

В классическом МНА каждая голова имеет свои проекции:

$$Q_i = XW_{Q,i}, \quad K_i = XW_{K,i}, \quad V_i = XW_{V,i}$$

KV-кэш хранится отдельно для каждой головы.

Multi-Query Attention (MQA) предлагает оставить запросы многоголовыми, но использовать одни общие ключи и значения для всех голов:

$$Q_i = XW_{Q,i}, \quad K = XW_K, \quad V = XW_V$$

Разные головы могут задавать разные запросы, но читать информацию из одного общего набора ключей и значений. В этом случае объем KV-кэш уменьшается примерно в N_{heads} раз.

Но в этом случае модель теряет часть выразительности и качество модели снижается.

Компромиссный вариант между МНА и MQA — **Grouped-Query Attention** (GQA). В GQA головы разбиваются на группы. Внутри этих групп головы запросов используют общие головы ключей и значений:

$$K_g = XW_{K,g}, \quad V_g = XW_{V,g},$$

где g — номер группы. Если число групп равно числу голов — то получаем МНА. Если группа всего одна — то получаем MQA.

В этом случае объем KV-кэш уменьшится в $\frac{N_{\text{heads}}}{N_{\text{groups}}}$.

2.3 Многоглавое скрытое внимание

Многоглавое скрытое внимание (Multihead latent attention, MLA) было разработано DeepSeek для снижения затрат памяти KV-кэша без снижения качества модели, как при использовании MQA или GQA.

MLA добавляет дополнительный шаг в процесс получения матриц K и V из X . Данные предварительно сжимаются в скрытое пространство:

$$L_{KV} = XW_L,$$

а затем восстанавливаются

$$K = L_{KV}W_K, \quad V = L_{KV}W_V$$

Казалось бы, на этом всё. Однако многоголовое внимание теперь можно упростить так, чтобы избежать лишних вычислений:

$$\begin{aligned} S &= QK^\top = XW_Q(L_{KV}W_K)^\top = X \underbrace{W_QW_K^\top}_{\widetilde{W}_Q} L_{KV}^\top = X\widetilde{W}_Q L_{KV}^\top, \\ O &= \begin{bmatrix} O_{h_1} \\ O_{h_2} \\ \dots \\ O_{h_n} \end{bmatrix}^\top \quad W_O = \begin{bmatrix} A_1V_1 \\ A_2V_2 \\ \dots \\ A_nV_n \end{bmatrix}^\top \quad W_O = \begin{bmatrix} A_1L_{KV}W_{V,1} \\ A_2L_{KV}W_{V,2} \\ \dots \\ A_nL_{KV}W_{V,n} \end{bmatrix}^\top \quad W_O = \\ &= \begin{bmatrix} A_1L_{KV} \\ A_2L_{KV} \\ \dots \\ A_nL_{KV} \end{bmatrix}^\top \quad \underbrace{\text{diag}\left(\{W_{V,i}\}_{i=1}^n\right)}_{\widetilde{W}_O} W_O = \begin{bmatrix} A_1L_{KV} \\ A_2L_{KV} \\ \dots \\ A_nL_{KV} \end{bmatrix}^\top \quad \widetilde{W}_O \end{aligned}$$

Итого, MLA можно записать так:

$$\widetilde{Q} = X\widetilde{W}_Q, \quad \widetilde{W}_Q = W_QW_K^\top,$$

$$L_{KV} = XW_L,$$

матрица внимания вычисляется следующим образом:

$$S = \widetilde{Q}L_{KV}^\top, \quad A = \text{Softmax}\left(\text{Mask}\left[\frac{\widetilde{Q}L_{KV}^\top}{\sqrt{d}}\right]\right),$$

объединение голов:

$$O = \begin{bmatrix} O_{h_1} \\ O_{h_2} \\ \dots \\ O_{h_n} \end{bmatrix}^\top \quad \widetilde{W}_O, \quad O_{h_i} = A L_{KV}, \quad \widetilde{W}_O = \text{diag}\left(\{W_{V,i}^\top\}_{i=1}^n\right) W_O$$

MLA позволяет достичь уменьшения объёма KV-кэша на 93% не просто без падения качества модели, а даже с небольшим улучшением. Однако, такой подход плохо работает на малых моделях, а потому наиболее эффективен в LLM.

2.4 Разреженное внимание

Оригинальный механизм внимания учитывает все связи всех токенов со всеми. В маскированном внимании в декодировщике учитываются все связи токенов со всеми предыдущими, но игнорируются связи со следующими токенами.

Если визуализировать матрицы внимания в обученной модели, то можно заметить, что значительная часть элементов имеет малый вклад. Наиболее сильными оказываются локальные связи между токенами, некоторые глобальные связи и отдельные дальние зависимости.

Шаблонное разреженное внимание В модели Sparse Transformer была предложена идея шаблонного разреженного внимания. В частности использовались два паттерна: блочно-диагональное внимание (внимание считается блоками вдоль главной диагонали) и скользящее внимание — внимание к токенам через фиксированный шаг по последовательности.

Вычисление внимания лишь для некоторых блоков хорошо сочетается с тайлингом матриц на GPU.

Дальнейшее развитие было представлено в модели BigBird. Эта модель объединяет несколько шаблонов: глобальные связи, связи ближайших токенов и случайные связи. Глобальные токены имеют полное внимание ко всем токенам. Глобальными токенами могут быть, например, первые токены последовательности или специальные токены.

Локальные связи позволяют токенам взаимодействовать с ближайшими соседями.

Случайные связи формируются по случайному шаблону разрежения. Они помогают передвигать информацию между удаленными частями последовательности.

Однако при вычислении подобных элементов на GPU нарушается согласованный доступ к памяти. Поэтому имеет смысл вычислять не просто случайные элементы, а случайные блоки с применением тайлинга.

Благодаря комбинации локальных, глобальных и случайных связей любой токен способен влиять на любой другой через некоторое количество шагов. Авторы BigBird в статье “BigBird: Transformers for Longer Sequences” теоретически показали, что разреженное внимание может быть таким же выразительным, как и полное.

Скользящее внимание Идея, что токен может влиять на каждый другой токен не напрямую, а через некоторое количество шагов, то есть через несколько слоёв внимания, нашла реализацию в идее **скользящего оконного внимания** (sliding window attention).

Каждый токен взаимодействует только с токенами фиксированного окна длиной $2w + 1$, где w — “радиус” окна, w токенов слева и w токенов справа. Это окно сдвигается на каждом новом слое. Чтобы не потерять глобальные зависимости, вводятся глобальные токены аналогично BigBird.

Вычислительная сложность такого механизма внимания линейна по длине $O(w \times \text{seq_length})$.

Обуемое разреженное внимание Шаблон связей в шаблонном разреженном внимании задается заранее и не зависит от входных данных. Однако, разные токены могут требовать разного набора связей.

Так, например, внутри одной головы для одних токенов важнее локальный контекст, а для других — дальние зависимости. А что если не фиксировать структуру внимания, а позволить модели самой выучивать шаблон, какие связи сохранять.

Такой механизм называется **обуемым разреженным вниманием**.

Разреженное внимание DeepSeek Чтобы получить обуемое разреженное внимание, необходим механизм, который позволит быстро оценивать релевантность каждого токена, чтобы оставить только самые важные для вычисления механизма внимания.

Исследователи из DeepSeek предложили инструмент Lightning Indexer. Он вычисляет оценку между текущим вектором запроса и предыдущим токеном. Эта оценка помогает модели решить, на какие предыдущие токены должен обратить внимание следующий запрос:

$$I^{t,s} = \sum_{j=1}^{n_h} w_j^t \text{ReLU}(q_j^t k_j^s)$$

То есть мы по сути перед вычислением внимания мы вычисляем его приближенную оценку, которая поможет нам определить, какие связи важны на текущем слое.

Поскольку информация о позиции токенов q_j^t и k_j^s будет полезной, добавим позиционное кодирование RoPE.

Индексатор, чтобы он работал корректно, необходимо обучить. На первой стадии заморозим слой внимания и будем обновлять только параметры индексатора.

То есть здесь внимание остается плотным. Из этого плотного внимания получают целевое распределение p^t : для текущего токена берутся веса из матрицы A или S , суммируются по головам и нормализуются вдоль последовательности.

Индексатор обучается так, чтобы его распределение было похоже на распределение плотного внимания:

$$L^I = \sum_t D_{KL} \left\langle p^{t,\cdot} \parallel \text{Softmax}(I^{t,\cdot}) \right\rangle$$

Во второй стадии обучения веса внимания размораживаются. Теперь для каждого токена индексатор выбирает множество из k самых важных позиций:

$$s^t = \text{Top}_k(I^{t,\cdot})$$

Модель теперь должна адаптироваться к тому, что часть связей внимания удалена, поэтому и индексатор необходимо переучивать под новое распределение:

$$L^I = \sum_t D_{KL} \langle p^{t,s^t} || \text{Softmax}(I^{t,s^t}) \rangle$$

Такой индексатор требует дополнительных вычислений, синхронизаций и, возможно, даже коммуникаций при обучении на длинном контексте.

Например, вычисление $\text{Softmax}(I^{t,\cdot})$ потребует по крайней мере сбора всех $I^{t,\cdot}$ на одном девайсе для выполнения редукций. А это значит, что при использовании контекстного параллелизма и параллелизма последовательностей, потребуются дополнительная коммуникация.

Эффективность такого индексатора напрямую зависит от того, насколько дешево он устроен и насколько хорошо его вычисления реализованы на GPU. Например, можно использовать более низкую точность и квантизацию, но об этом чуть позднее.

Такой механизм разреженного внимания называется **разреженное внимание DeepSeek (DeepSeek Sparse Attention, DSA)**

2.5 Сжатое внимание: CSA, HCA

Нам не всегда хотелось бы отбрасывать токены последовательности во время фильтрации. Да, полезной информации в них немного, но она всё же она есть.

Попробуем сжимать информацию, которая поступает на вход механизма внимания.

Пусть $H \in \mathbb{R}^{n \times d}$ — матрица входных токенов, к которым мы хотим применить механизм внимания. Аналогично MLA, сожмём эти входы в пространство меньшей размерности:

$$C = HW^{KV},$$

где $C \in \mathbb{R}^{n \times d_c}$, а размерность d_c меньше размерности d .

Мы сжали матрицу H по оси d . Теперь сожмём её по оси n . Возьмём 4 соседних токена и объединим их в один вектор. Самый простой способ — усреднить эти вектора:

$$c_1^{\text{comp}} = 0.25c_1 + 0.25c_2 + 0.25c_3 + 0.25c_4$$

Однако, такой способ не идеален. Позволим модели самой выбирать, с какими весами усреднять токены. Но простые скалярные веса не будут зависеть от входных данных. Зададим их следующим образом:

$$S = \text{Softmax}(HW^Z)$$

И теперь усредним:

$$c_1^{\text{comp}} = \sum_{i=1}^4 s_i c_i, \quad c_2^{\text{comp}} = \sum_{i=5}^8 s_i c_i, \quad \dots$$

Но вектора $c \in C$ имеют довольно высокую размерность, например $d_c = 512$. Усреднять такие вектора с помощью скалярных коэффициентов будет неэффективно. Поэтому обобщим эти коэффициенты до векторов размерности d_c . Тогда:

$$S^\top = \text{Softmax}\left[(HW^Z)^\top\right], \quad S \in \mathbb{R}^{n \times d_c},$$

запись здесь транспонирована, поскольку Softmax необходимо брать по столбцам матрицы HW^Z .

Далее:

$$c_1^{\text{comp}} = \sum_{i=1}^4 s_i \odot c_i, \quad c_2^{\text{comp}} = \sum_{i=5}^8 s_i \odot c_i, \quad \dots,$$

где $\odot$ — оператор покомпонентного умножения.

Однако при таком сжатии мы создаём искусственные границы между группами токенов. Поэтому сделаем эти группы пересекающимися следующим образом:

$$c_1^{\text{comp}} = \sum_{i=1}^4 s_i \odot c_i, \quad c_2^{\text{comp}} = \sum_{i=1}^8 s_i \odot c_i, \quad c_2^{\text{comp}} = \sum_{i=5}^1 2s_i \odot c_i, \quad \dots$$

Для этого также разделим матрицы W^{KV} и W^Z на матрицы для текущей группы и для прошлой группы: W_{curr}^{KV} , W_{prev}^{KV} и W_{curr}^Z , W_{prev}^Z соответственно.

Получаем:

$$c_1^{\text{comp}} = \sum_{i=1}^4 s_{\text{curr},i} \odot c_{\text{curr},i}, \quad c_2^{\text{comp}} = \sum_{i=1}^8 s_{\text{curr},i} \odot c_{\text{curr},i}, \quad \dots$$

Такое сжатие называется **сжатием на уровне токенов**.

Но почему мы вычисляем

$$S_{\text{curr}}^\top = \text{Softmax}\left[(HW_{\text{curr}}^Z)^\top\right],$$

а не

$$S_{\text{curr}}^\top = \text{Softmax}\left[(C_{\text{curr}}W_{\text{curr}}^Z)^\top\right],$$

и аналогично для S_{prev} ?

Дело в том, что в первом варианте мы можем вычислять матрицы S_{curr} , S_{prev} , C_{curr} и C_{prev} параллельно, что позволяет проводить вычисления в одном кернеле.

Во втором случае нам придется сначала в одном ядре сначала вычислить матрицы C_{curr} и C_{prev} , а уже следом во втором — S_{curr} и S_{prev} .

Время интегрировать этот механизм сжатия в механизм внимания. Мы развиваем идею MLA, поскольку входы H проецируем в сжатое латентное пространство, к которому далее применяем сжатие на уровне токенов.

Получаем матрицу сжатых токенов

$$C^{\text{comp}} \in \mathbb{R}^{n/4 \times d_c}$$

Эта матрица поступает на вход всех K и V голов. Благодаря сжатию, каждая голова может использовать вектора k и v большей размерности. Однако, это сопряжено с дополнительными вычислительными трудностями.

Так, при увеличении размерности внутри головы внимания, матрица W^Q становится огромной:

$$\dim(W^Q) = d \cdot c \cdot h$$

Воспользуемся низкоранговым подходом. Сначала спроецируем входной вектор в латентное пространство с помощью матрицы $W_{\text{down}}^Q \in \mathbb{R}^{d \times r}$.

Затем получим Q из латентного пространства с помощью матрицы $W_{\text{up}}^Q \in \mathbb{R}^{r \times (c \times h)}$:

$$Q = H W_{\text{down}}^Q W_{\text{up}}^Q$$

Аналогичная проблема возникает и с выходной матрицей $O \in \mathbb{R}^{n \times (c \times h)}$. Разделим эту матрицу на g групп $\{O_i\}_{i=1}^g$, где

$$O_i \in \mathbb{R}^{n \times \left(\frac{c \times h}{g}\right)}$$

Спроецируем каждую такую группу в пространство большей размерности с помощью матрицы $W_{\text{down}}^O \in \mathbb{R}^{\left(\frac{c \times h}{g}\right) \times n_g}$:

$$O_i^G = O_i W_{\text{down}}^O, \quad O_i^G \in \mathbb{R}^{n \times n_g}$$

После этого матрицы $\{O_i^G\}_{i=1}^g$ конкатенируем в матрицу O^G и проецируем в пространство исходной размерности с помощью матрицы $W_{\text{up}}^O \in \mathbb{R}^{(n_g \times n) \times d}$.

Этот подход называется **группировкой выходных проекций (Grouped Output Projection)**. Таким образом мы можем сократить количество необходимых параметров для матриц W^Q и W^O почти в 6.5 раз.

Вместе с механизмом сжатия токенов и группировкой выходных проекций мы также можем использовать DSA. Объединение этих методов с механизмом многоглавого внимания называется **сжатым разреженным вниманием (Compressed Sparse Attention, CSA)**.

CSA сжимает 4 вектора в 1. Однако, DeepSeek представили также механизм сильносжатого внимания (Heavily Compressed Attention, HCA), который позволяет сжать 128 векторов в 1. При таком сильном сжатии даже нет необходимости в использовании разреженного внимания.

Использование таких механизмов позволяет достичь размера контекстного окна в миллион токенов.

3 Обучение в пониженной точности

3.1 Смешанная точность

Вычисления в низкой точности — это способ значительно ускорить вычисления и уменьшить потребление памяти.

Однако, низкая точность снижает стабильность обучения. При этом, некоторые элементы модели значительно чувствительнее к понижению точности, чем другие. Поэтому давайте хранить и считать разные модули в разных численных форматах.

Такой подход называется **смешанной точностью**. Так например, матричное умножение мы можем вычислять в bf16, fp8 или fp4 типах, а вот аккумуляцию необходимо выполнять в более высокой точности.

Чувствительные операции, такие как нормализации, Softmax и функции активации имеет смысл выполнять в более высокой точности, чтобы была меньшая погрешность в вычислениях.

3.1.1 Мастер-веса

Мы можем хранить некоторые веса в пониженной точности и обучать их напрямую. Но у такого подхода есть проблема: обновления параметров могут быть слишком малыми относительно точности представления данных.

Малое обновление может просто потеряться при округлении и некоторые веса не изменятся.

Чтобы избежать такой проблемы, используются **мастер-веса** — это копия параметров в более высокой точности, например, в fp32.

Для прямого и обратного распространения всё ещё используется копия весов в низкой точности, например, в fp8. Градиенты тоже вычисляются в низкой точности. Но шаг оптимизатора применяется к мастер-весам. После этого веса, на которых проводятся вычисления, обновляются до версии мастер-весов.

3.1.2 Loss-scaling

В fp16 и особенно в bf16 типах маленькие значения градиентов могут стать нулевыми из-за округления. Это приводит к той же проблеме, которую мы только что решали добавлением мастер-весов: параметры перестают обновляться.

Чтобы исправить это, используют **масштабирование функции потерь (loss-scaling)**. Перед обратным распространением умножим функцию потерь на большой коэффициент:

$$\tilde{L} = sL,$$

где s — коэффициент масштабирования.
Тогда градиенты тоже масштабируются:

$$\frac{\partial \tilde{L}}{\partial w} = s \frac{\partial L}{\partial w}$$

После вычисления градиентов их необходимо вернуть к исходному масштабу. Но деление мы будем выполнять уже в более высокой точности при обновлении мастер-весов:

$$g = \frac{\tilde{g}}{s}$$

Масштабирование функции потерь бывает статическим и динамическим.

При статическом масштабировании коэффициент s задается заранее. Это просто, но не всегда эффективно. Если s слишком мал, то он не решает проблему округления. Если коэффициент s слишком большой, то появляется риск переполнения.

При динамическом масштабировании коэффициент подбирается во время обучения. Если переполнений нет, коэффициент масштабирования можно постепенно увеличивать. Если появились `inf` или `nan` в градиентах — масштаб необходимо уменьшить. Шаг оптимизатора в этом случае пропускается.

3.2 Квантизация

Мы можем представить веса, активации и (при обучении) градиенты с помощью еще более меньшего числа бит и целочисленном типе с помощью **квантизации**. Это позволит значительно экономить память.

Однако квантизация создает компромисс — чем меньше бит используется, тем сильнее ошибка в представлении чисел.

Рассмотрим тензор x , который хотим представить в целочисленном формате:

$$x \approx sq,$$

где s — масштаб, а q — целочисленное представление.

3.2.1 Симметричная и асимметричная квантизация

Квантизация бывает **симметричной** и **асимметричной**.

В симметричной квантизации ноль вещественного тензора соответствует нулю целочисленного тензора:

$$q = \text{round}\left(\frac{x}{s}\right)$$

Масштаб s выбирается таким образом, чтобы покрыть диапазон значений тензора:

$$s = \frac{\max |x|}{q_{\max}}$$

В асимметричной квантизации добавляется смещение:

$$q = \text{round}\left(\frac{x}{s}\right) + z$$

Тогда восстановление выглядит так:

$$\hat{x} = s(q - z)$$

Симметричная квантизация эффективна при работе с симметричными тензорами. Например, с весами.

Асимметричную квантизацию полезно применять там, где распределение значений, например, всегда положительно или отрицательно. Например — для некоторых активаций. Но такая квантизация сложнее в реализации, ведь помимо масштаба необходимо также учитывать и смещение.

3.2.2 Блочная квантизация

Одна из самых главных проблем квантизации — выбросы. Предположим, что почти все значения тензора лежат в диапазоне $x \in [-1, 1]$, но есть несколько элементов с большой величиной $x_j = 20$.

Если масштаб выбирается по максимальному значению, то он должен покрыть диапазон до 20. Тогда остальные значения будут представлены слишком грубо.

В трансформерах выбросы особенно заметны в активациях. Наивная квантизация с одним масштабом часто даёт сильную деградацию качества. Поэтому необходимо брать локальные коэффициенты масштабирования: по строкам или по блокам.

В **квантизации по строкам (row-wise quantization)** масштаб выбирается отдельно для каждой строки:

$$s_i = \frac{\max_j |x_{ij}|}{q_{\max}}$$

Тогда:

$$q_{ij} = \text{round}\left(\frac{x_{ij}}{s_i}\right)$$

В **блочной квантизации** тензор разбивается на блоки и масштаб вычисляется для каждого блока по отдельности. Чем меньше блок, тем точнее

квантизация. Но тем больше необходимо хранить в памяти коэффициентов масштабирования.

Блочную квантизацию можно рассматривать как компромиссный вариант между разными шагами точности.

Блочная квантизация и квантизация по строкам особенно важны при обучении в очень низкой точности: fp8 или fp4. Так, мы можем значительно удешевить обучение модели с меньшим риском развала обучения.

Квантизация также применима в коммуникациях: например, уменьшив точность с bf16 до fp8, мы можем значительно уменьшить давление на коммуникации при выполнении AllGather в FSDP.

При вычислении, веса или активации деквантизируются на лету в более высокую точность. Причем мы можем сделать это в одном кернеле с основными вычислениями. По окончании вычислений они могут быть квантизованы обратно в том же кернеле.